Troubleshooting — symptom-indexed reference

The **symptom-first** companion to `om-runbooks.md`. The runbooks are indexed by *task* ("rotate the certificate", "respond to an alarm"); this document is indexed by *what you are looking at* — an error string, an alarm name, an empty dashboard, a developer's support ticket.

How to use it. Find the section for the subsystem the symptom points at, match the symptom text, read the cause, then apply the fix. Where a full procedure already exists, the entry gives symptom → cause and then names the runbook section that has the steps — it does not repeat them. Cross-document pointers used throughout:

- `om-runbooks.md` — rotations, updates, alarm response, backup/restore, Bedrock prompt logging, CMK re-adoption, teardown.
- `greenfield-deployment.md` — first deploy, in order, including the org prerequisites that gate it.
- `cost-controls.md` — spend enforcement, the dashboard, the fail-closed spend-store incident runbook.
- `client-config.md` — Part I is the developer-facing user manual (its §5 is the end-user troubleshooting list); Part II is the managed-settings enforcement model.

Placeholders: `<prefix>` is `NAME_PREFIX` / the templates' `NamePrefix`, `<gateway-fqdn>` is `GATEWAY_FQDN`, `<region>` is `AWS_REGION`. Every variable named below is a `deploy.env` variable, a CloudFormation parameter, or an environment variable in a template — none are org-specific values.

Sections

---

1. Stack deploys & CloudFormation

1.1 Deploy fails: log group "already exists"

Symptom. A deploy into an account that has run this deployment before fails creating `/aws/rds/instance/<prefix>-store/postgresql`, `/aws/lambda/<prefix>-db-bootstrap` or `/aws/lambda/<prefix>-db-rotation`.

Cause. Every log group in every stack carries `DeletionPolicy: Retain` (logs deliberately outlive stacks) and the groups have fixed names, so they survive a teardown and collide on re-create. The templates also *pre-create* those three groups — the only way the CMK and the retention setting apply — so an account where RDS or Lambda auto-created them under the same names collides too.

Fix. Export anything still needed, then delete the groups before deploying; the command list is in `greenfield-deployment.md` Phase 1. The same applies to the `/ecs/*` and `/claude/*` groups before a full re-create from scratch. Where a service auto-created the group, delete it and let the template create and own it.

1.2 Deploy fails: "version 16.x does not exist for postgres"

Symptom. `deploy-database.sh` fails immediately on the engine version.

Cause. The template's default minor version is not offered in this region.

Fix. List what the region offers and pin `DB_ENGINE_VERSION` in `deploy.env`:

```
aws rds describe-db-engine-versions --engine postgres --region "$AWS_REGION" \
  --query "DBEngineVersions[?starts_with(EngineVersion,'16.')].EngineVersion" \
  --output text
```

1.3 The `Custom::DbAppUserBootstrap` resource hangs, then fails

Symptom. The gateway (02) deploy sits on the bootstrap custom resource and fails after about five minutes.

Cause. Almost always a VPC-Lambda reachability gap — a security-group or VPC-endpoint path to Postgres or Secrets Manager — not the SQL. The resource carries `ServiceTimeout: 300` so a stuck bootstrap fails the stack in minutes instead of CloudFormation's hour-long default.

Fix. Read `/aws/lambda/<prefix>-db-bootstrap` — the function logs every step and holds the real cause. The CloudFormation event does not. Check the endpoint security group's ingress (§3.4) before anything else.

1.4 Targets never go healthy

Symptom. ECS tasks start and stay up, but `describe-target-health` reports `unhealthy` indefinitely.

Cause. ALB health checks default to plain **HTTP** regardless of the target group's protocol. The targets serve TLS, so a plaintext probe never succeeds and no target is ever registered healthy.

Fix. `HealthCheckProtocol: HTTPS` must be set explicitly on the target group. The templates set it — a locally modified template is the first suspect. After that, check the per-task TLS certificate path in the task logs.

1.5 The stack sits in `CREATE_FAILED` / `UPDATE_FAILED`

Not a fault to clean up. `deploy-gateway.sh` runs with `--disable-rollback` by default, so a failed deploy **keeps its healthy resources**: fix the cause and re-run the script, and the deploy continues from the failed resource. This is deliberate — a full rollback costs roughly half an hour of db-admin Lambda ENI teardown and cannot delete the deletion-protected ALB, which wedges the stack in `DELETE_FAILED`. `CFN_DISABLE_ROLLBACK=false` restores classic auto-rollback if it is ever wanted.

1.6 `VAR=value ./scripts/deploy-*.sh` appears to have no effect

Cause. `common.sh` sources `deploy.env` **after** the process environment is set, so the file's exported value wins for every variable the file defines. A command-line prefix is silently overwritten.

Fix. Edit `deploy.env` (or use `set_env_var`). This bites most often with `TELEMETRY_FAIL_CLOSED` and `KMS_KEY_ARN`.

1.7 A rebuilt image "deploys" but the old code keeps running

Symptom. Either the push is rejected outright, or the stack update succeeds and behaviour is unchanged.

Cause. ECR repositories are IMMUTABLE. A same-tag rebuild cannot be pushed, and an unchanged image URI leaves the service or Lambda on the old image.

Fix. Always bump the tag — `IMAGE_TAG`, `GRAFANA_IMAGE_TAG`, `PORTAL_VERSION`, `DBADMIN_VERSION` — and push **before** the stack update that expects the new content (om-runbooks §6). See §14.4 for the wrong-architecture variant of this trap.

1.8 Update fails: "Cannot update export … in use"

Cause. While a downstream stack imports an export, the upstream stack cannot change that export's value. An apparently trivial edit is enough — a security group's `GroupDescription`, for instance, forces replacement and so changes the exported id.

Fix. There is no in-place path: the downstream stack must be deleted first. Treat exported values (the CMK, the DB endpoint, the master-secret ARN, the client SG from 01; the SGs, listener and cluster from 02) as day-one decisions.

1.9 A `DeletionPolicy` change appears not to apply

Cause. A CloudFormation change touching **only** `DeletionPolicy` or `UpdateReplacePolicy` on an already-deployed resource can be dropped as a no-op, silently leaving the old policy in force.

Fix. The affected resources carry a load-bearing tag (`retention-policy`) so the update is a real property diff. **Do not remove those tags.** After such an update, confirm the tag is visible on the live resource — that is the proof the update was not a no-op.

1.10 An interface VPC endpoint fails to create

Symptom. A stack fails creating an interface endpoint that carries a `PolicyDocument`, or fails because a private hosted zone already exists for that service.

Cause. Two distinct region/landing-zone constraints: not every service supports endpoint policies in every region (the `ecs` endpoint deliberately carries none for this reason), and creating a local endpoint with `PrivateDnsEnabled: true` fails when a shared-services spoke already centralizes that endpoint with an associated private hosted zone.

Fix. Pre-check policy support before adding a policy:

```
aws ec2 describe-vpc-endpoint-services --region us-gov-west-1 \
  --service-names com.amazonaws.us-gov-west-1.<svc> \
  --query 'ServiceDetails[].VpcEndpointPolicySupported'
```

`false` → omit the policy and rely on IAM-side scoping. For the private-DNS conflict, use the centralized endpoint — and record that doing so drops the **endpoint-policy guardrail** the design otherwise relies on.

---

2. KMS & encryption

2.1 Key-policy changes apply to nothing; `alias/<prefix>` has vanished

Symptom. A statement added to the CMK's `KeyPolicy` in `01-database.yaml` never appears on the real key; the alias is gone; a downstream feature that depends on the statement (Bedrock prompt logging is the usual one) fails with an unrelated-looking error.

Cause. The database stack creates the CMK and persists its ARN into `deploy.env` as `KMS_KEY_ARN` for the other consumers (ECR encryption, mirroring, stacks 02/03/04). Feeding that same value back as the stack's own `KmsKeyArn` **parameter** flips a created-key deployment into bring-your-own mode: CloudFormation drops the `Retain`ed key from the stack, deletes the alias, and the stack never manages the key policy again.

Fix. `resolve_kms_param` in `common.sh` makes an existing stack keep its own recorded `KmsKeyArn` parameter rather than trusting `deploy.env`; `ALLOW_KMS_PARAM_CHANGE=1` is the explicit, named override for a deliberate key change; an unexpected `describe-stacks` error is fatal rather than ownership-flipping. `deploy-observability.sh` preflights the key policy and fails fast with the real cause.

Diagnosis on an affected deployment. Three checks together confirm it: the stack's `KmsKeyArn` parameter is non-empty; the key it names carries the stack's own description; the stack no longer lists the `KmsKey` / `KmsKeyAlias` resources.

```
aws cloudformation describe-stacks --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME" \
  --query "Stacks[0].Parameters[?ParameterKey=='KmsKeyArn'].ParameterValue" \
  --output text
```

Empty = stack-managed. An ARN = bring-your-own **or** detached.

Repair. Apply any missing statement out-of-band (get → append → put) to unblock immediately, then optionally re-adopt the key via a CloudFormation IMPORT changeset — the full procedure, including the two-phase import and the post-import drift check, is **om-runbooks §11a**. Two things that procedure exists to prevent: a one-shot import that also flips `KmsKeyArn` is rejected, and **import does not apply template properties**, so a same-template re-run afterwards is a genuine no-op.

2.2 A caller gets a server-side 403 against a CMK-encrypted resource

Symptom. Fast (tens of milliseconds), server-sourced 403s where connectivity is obviously fine, or an `AccessDenied` naming a KMS action.

Cause. A service grant on the key covers only that service's *internal* use. The **caller's** own IAM role must hold the KMS action for a data-plane call. Three places this shows up in this deployment:

Caller	Needs	Symptom section
Grafana querying AMP	`kms:Decrypt`	§7.3
Telemetry sidecar remote-writing to AMP	`kms:GenerateDataKey`	§7.3
Operator reading a CMK-encrypted secret	`kms:Decrypt`	§5.6

Fix. Grant the action on the CMK, scoped with a `kms:ViaService` condition for the fronting service (`aps.<region>.amazonaws.com`, `secretsmanager.<region>.amazonaws.com`). Check for this before chasing the service's own resource policy.

2.3 Encryption choices cannot be changed later

Encryption-at-rest is a **day-one decision** in several places, and the symptom of getting it wrong is discovered long after:

- Changing the RDS storage CMK on an existing deployment requires snapshot → tear down downstream stacks → rebuild → restore, not an update.
- Enabling the AMP CMK on an existing workspace **replaces the workspace** and orphans metric history.
- ECR repositories created before the CMK existed stay SSE-S3 forever — encryption is fixed at repository creation. Recreate them if CMK coverage is mandatory. This is why the database stack (which creates the CMK) is deployed **first**.

---

3. Network, TLS & DNS

3.1 ALB access-log enablement fails `AccessDenied`

Symptom. A stack create or update fails validating ALB access logging against a bucket policy that reads correct — intermittently at first, then consistently.

Cause — check in this order.

1. **A landing-zone auto-remediation** independently rewriting the ALB's access-log configuration to point at a central logging bucket, fighting the template. Suspect this *before* the bucket policy whenever the policy reads correct.
2. **A create-time race.** ELB's create-time test write can land before a just-created bucket policy is live, and CloudFormation does not retry.
3. `BucketOwnerEnforced`**.** ACL-based delivery by the legacy per-region ELB account fails when ACLs are disabled unless the bucket policy also grants the `logdelivery.elasticloadbalancing.amazonaws.com` **service principal**.

Fix. For (1), get the ALB exempted from the automation, or adopt the central bucket — do not re-edit the bucket policy. For (3), the template already grants **both** delivery principals: in this region the writer may authenticate as either, and `BucketOwnerEnforced` accepts the legacy writer's `bucket-owner-full-control` canned ACL, so ACLs stay disabled.

3.2 Long inference turns truncate mid-stream

Symptom. A truncated SSE response or a connection reset on extended-thinking turns or long tool results; also on every ECS rolling deploy.

Cause. The ALB idle timeout closes a connection that carries no bytes for its duration, and a long turn can be silent for minutes. Separately, the target-group deregistration delay cuts in-flight streams older than the delay on every deploy.

Fix. `ALB_IDLE_TIMEOUT_SECONDS` (parameter `AlbIdleTimeoutSeconds`, default 900) and `DEREGISTRATION_DELAY_SECONDS` (parameter `DeregistrationDelaySeconds`, default 300) size these. Raising the deregistration delay is a trade: the full delay is always waited, so every deploy takes that much longer.

3.3 A no-NAT spoke deploy fails with `CannotPullContainerError`

Cause. Expected when the supporting VPC endpoints are not created and there is no confirmed transit-gateway path to central NAT with the ECR/S3 domains allowlisted.

Fix. Decide explicitly per deployment: create the supporting endpoints, or confirm the inspected egress path. There is no third option.

3.4 An in-VPC caller's AWS API calls black-hole

Symptom. A downstream stack rolls back at image pull or secret read; an operator's in-VPC admin or build host "can't reach AWS" while everything else works.

Cause. Interface endpoints' **private DNS captures AWS API calls from every client in the VPC**, not only the workloads the endpoints were created for. Any in-VPC caller of ecr / logs / secretsmanager / ecs — including an admin host and tasks created by a *different* stack — must appear in the endpoint security group's ingress, or its calls silently black-hole.

Fix. The endpoint SG is exported from the stack that owns it; add an ingress rule for every consumer, including the parameterized admin-host SG. No static gate catches this — it is a semantic cross-stack reachability property — so re-check it by hand in any change to the endpoint SG or a task SG.

3.5 A CLI helper's TLS verification fails *because* the corporate CA was added

Symptom. A helper that "adds" the corporate CA still fails verification, and which requests fail flips depending on the path taken.

Cause. `curl --cacert` **replaces** curl's trust store rather than extending it. Handing curl exactly one extra CA breaks verification whenever the chain actually presented terminates in the *other* trusted root — internal PKI versus the TLS-inspection root.

Fix. The `combined_ca_bundle` helper in `common.sh` builds the system store **plus** `GATEWAY_CA_BUNDLE` **plus** `EXTRA_CA_CERT_PATH` into one mode-600 temporary bundle; `set-spend-limit.sh` uses it. **Never** `-k`**.** On a persistent failure the script prints the exact `openssl` command to compare the presented issuer against the bundle. The same class applies to any container that must dial the gateway ALB (the portal's fingerprint and usage calls): it needs **both** the enterprise CA and the inspection CA staged into its image trust store.

3.6 DNS assertions disagree with reality from a laptop behind ZPA

Symptom. `verify-gateway.sh` DNS checks pass when the corporate CNAME does not exist, or AAAA checks false-fail.

Cause. Client Connector answers the app-segment FQDN with a **synthetic 100.64/10 address**; the real lookup happens at the **App Connector** using that host's resolvers. Synthetic answers are not authoritative, and ZPA's IPv6 synthetic ranges break naive AAAA assertions.

Fix. Run DNS assertions from an App Connector's resolution context — the script says so when it detects synthetic answers. What actually has to be true:

- Every App Connector can resolve the **corporate CNAME**.
- The CNAME target — the internal ALB's `internal-*.elb.amazonaws.com` name — is a normal **public** record that returns private addresses from any resolver, so it needs no Resolver inbound endpoint, conditional forwarder or private hosted zone.
- **App Connectors inside an AWS VPC** use the `.2` resolver, which knows nothing of the corporate zone: add a Route 53 Resolver **outbound** rule forwarding that zone to AD DNS, or the connector gets NXDOMAIN and the user sees an unexplained ZPA timeout.
- A resolver or appliance that strips RFC1918 answers out of public-zone responses (anti-DNS-rebinding) breaks resolution of the ELB target. The escape hatch is a private hosted zone associated to the connector VPC, or a hosts entry.

3.7 The served certificate is the inspector's, not the gateway's

Symptom. `verify-gateway.sh` hard-fails on an inspection-issued certificate, or a naive fingerprint check prints a value that does not match what was imported into ACM.

Cause. TLS inspection in front of the gateway FQDN replaces the leaf, which breaks the client's certificate-fingerprint pin — the whole trust-on-first-use control — and would publish the **inspection intermediate's** fingerprint as the pinning value.

Fix. The client-side entry for the gateway FQDN (ZPA app segment on TCP 443, or a ZIA SSL-inspection exemption plus app bypass) must be active. There is no client-side workaround, and disabling TLS verification on the client is never the answer. `verify-gateway.sh` cross-checks the served leaf's SHA-256 against the certificate imported into ACM for exactly this reason.

3.8 A lab certificate fails TLS before the fingerprint prompt appears

Cause. Claude Code validates the chain **first** and pins the fingerprint second, so an untrusted leaf fails before the prompt is drawn.

Fix. Trust the leaf on the client — a `Cert:\CurrentUser\Root` import (no admin) or the installer's `-ExtraCaCertPath`, which writes `NODE_EXTRA_CA_CERTS`; the same mechanism carries a real enterprise CA. A self-signed leaf is its own trust anchor, so the leaf itself goes into the store — there is no separate CA to import. Never publish a lab fingerprint as production-trusted.

3.9 Certificate rotation re-triggers every developer's trust prompt

Cause. Rotation changes the leaf, so Claude Code's first-connect trust prompt returns fleet-wide.

Fix. Publish the new SHA-256 fingerprint **before** cutting over, and rotate **in place** under the same ACM ARN so the ALB keeps its DNS name and listener configuration. Never delete-and-recreate the certificate or edit the listener's `CertificateArn`. Full procedure: om-runbooks §1.

3.10 Proxy plumbing: the two things that silently break egress

- **The proxy hostname must resolve from the VPC.** If `HttpsProxyUrl` is a corporate name, the VPC resolver needs a Resolver outbound rule for that zone — otherwise the gateway cannot reach its proxy and Okta login breaks the moment the proxy is configured.
- `NO_PROXY` **must be scoped to the exact internal namespace** (`.<prefix>.internal`), not a bare `.internal`, which would also bypass the proxy for corporate `*.internal` zones. Any container that must talk to the gateway FQDN directly needs that FQDN in `NO_PROXY` so the call does not chase the egress proxy.

---

4. Identity & OIDC

4.1 Gateway boot fails at OIDC discovery — a TLS error, then `403 Forbidden`

Symptom. The gateway container never becomes healthy. Logs show a TLS verification error against the Okta issuer; once the certificate is trusted, they show *"expected 200 OK, got: 403 Forbidden"*.

Cause. Server-originated egress from the VPC through the inspection proxy fails in two independent halves: SSL inspection presents a derived certificate (the TLS error), **and** policy refuses the identity-less server-originated request (the 403). The client-side entry for the gateway FQDN does **not** cover this — it is a separate rule on a separate path.

Fix. For the location carrying the workload VPC's central egress, request both an **ALLOW** for the Okta issuer FQDN on TCP 443 for server-originated traffic **and** an **SSL-inspection exemption** on the same path. The token exchange carries the OIDC client secret and should not transit inspection infrastructure. The template for this request is `docs/requests/networking-request-email.md`.

Interim fallback (TLS half only). Bake the inspection root CA into the image trust stores via `EXTRA_CA_CERT_PATH`. The policy ALLOW is required either way.

Distinguishing probe, from an in-VPC host:

```
curl -sv "https://<okta-issuer-host>/.well-known/openid-configuration"
```

An HTML block page = the policy ALLOW is missing. A JSON error body = an Okta network-zone 403 (Okta-side configuration). A TLS error = the inspection exemption is missing. Both halves in place = `200` + JSON.

4.2 Everyone is denied by Grafana or the portal; per-group spend caps match nobody

Symptom. Looks like an authorization bug: every user denied, the Grafana "Okta group" dashboard filter empty, group-scoped caps having no effect.

Cause. The Okta app has the `groups` **scope** but no groups **claim** configured on the authorization server. On an Okta *org* authorization server the scope alone yields group membership in neither the ID token nor `/userinfo`. Discovery metadata never lists claims, so this cannot be verified from metadata.

Fix. Okta-side: add a groups claim. Verify from a **real token** for a member user, not from discovery. `docs/requests/okta-request-email.md` is the request template. Cross-check from the data side with `scripts/diagnostics/dump-usage.sh`: `principal_emails` rows present but every `groups` value NULL or empty is the same diagnosis. Treat the groups claim as a hard operating prerequisite for any group-gated surface.

4.3 Grafana login fails in a way that looks like IdP misconfiguration

Cause. Grafana does not perform OIDC discovery — it derives the OAuth endpoint URLs from the issuer, and Okta **org** and **custom** authorization servers have different URL shapes (org: `<issuer>/oauth2/v1/…`, with the built-in `groups` scope; custom: `<issuer>/v1/…`). A trailing slash on the issuer is also rejected.

Fix. Match the endpoint shape to the authorization-server type actually in use before suspecting Okta.

4.4 Grafana: nobody can sign in, and the token exchange times out

Cause. With the local login form disabled, an Okta token exchange that cannot reach the network is a **total lockout**. In a landing zone where the egress proxy is mandatory, Grafana needs the same `HttpsProxyUrl` the gateway uses.

Fix. Pass the proxy to the Grafana task. The break-glass path is the bootstrap `admin` account, reachable only by redeploying with `GRAFANA_DISABLE_LOGIN_FORM=false`.

4.5 All logins fail immediately after rolling an Okta client secret

Cause. The new secret was written to Secrets Manager and the service rolled while Okta still expected the old one. With the Grafana login form disabled this is a total lockout.

Fix / **prevention.** Okta apps can hold two secrets during an overlap window: have the admin **add** the new secret without removing the old, confirm it is active, then roll. Recovery is to re-run the same `set-*-secret.sh` and paste the **previous** value. Full procedure: om-runbooks §2.

---

5. Database & secrets

5.1 Every DB connect fails: "self signed certificate in certificate chain"

Cause. The gateway's Postgres client **ignores the libpq** `sslrootcert=` **URL parameter** and verifies against the runtime's default trust store. RDS CAs are private, so verification always fails.

Fix. The RDS regional CA bundle is installed into the image's **OS trust store** (and exported via `NODE_EXTRA_CA_CERTS`, which extends rather than replaces the default roots); the connection URL carries `?sslmode=verify-full` only. Two consequences:

- `sslrootcert=` must not be re-added to the URL — the driver forwards unknown URL parameters to the server as session parameters, and Postgres rejects them ("unrecognized configuration parameter") at boot.
- An RDS CA change is an **image rebuild**, not a config flip: re-mirror the bundle on the egress host → transfer → rebuild **both** the gateway and db-admin images with bumped tags → deploy (om-runbooks §4). Rebuilding only one side leaves that side unable to validate once the CA cuts over. The db-admin Lambda itself is unaffected by the `sslrootcert=` behaviour — pg8000 takes an explicit CA file — but it still needs the current bundle baked in.

5.2 The first credential rotation looks like it did not happen

Cause. The app-secret rotation fires immediately at stack creation but is **asynchronous** — the stack goes green either way.

Fix. Verify rather than assume: confirm `AWSCURRENT` flipped to the alternate user.

```
aws secretsmanager get-secret-value --region "$AWS_REGION" \
  --secret-id "${NAME_PREFIX}/db-app-user" --query SecretString --output text \
  | jq -r .username
```

If it did not flip, tail `/aws/lambda/<prefix>-db-rotation` and check the `<prefix>-db-rotation-errors` alarm.

5.3 A rotation is failing — is the gateway about to lose its credential?

No. `<prefix>/db-app-user` alternates between `gateway_app` and `gateway_app_clone`; each rotation flips `AWSCURRENT` to the *other* user with a fresh password, so the **previous credential stays valid until the next rotation**. What is at risk is the rotation SLA, not availability.

Expected noise. At long rotation cadences the container-image Lambda is `Inactive` when rotation fires, so one failed invoke while Lambda re-optimizes is normal; Secrets Manager's own retries complete it. The `<prefix>-db-rotation-errors` alarm threshold deliberately tolerates that one expected error per scheduled rotation.

Triage. Tail `/aws/lambda/<prefix>-db-rotation` for the failing step of the four (`createSecret` / `setSecret` / `testSecret` / `finishSecret`), fix the cause, and re-drive with `rotate-secret`. To abandon an in-flight pending version, remove the `AWSPENDING` stage from it — the live credential is untouched. **Never hand-edit the secret**: it is Lambda-managed, and hand-edits desync it from the Postgres role passwords. Full procedure: om-runbooks §3.

Separately: the **RDS master** secret's managed weekly rotation affects no running workload. The gateway connects only as the least-privilege application user; the master secret is break-glass and no task injects it.

5.4 A consumer is locked out right after an unrelated deploy

Cause. Secrets that are set out-of-band (`OktaClientSecret`, `GrafanaOidcClientSecret`, `PortalOidcClientSecret`) hold a **placeholder** `SecretString` literal in the template, and `DbAppUserSecret` holds a managed value. Editing that literal — or the resource's `Name` or `Description` — triggers an update to the secret resource and re-applies the placeholder over the live value.

Fix. Do not edit those resources. Set and rotate those values only via the `set-*-secret.sh` scripts; the real value lives only in Secrets Manager.

5.5 A rotated generated key has no effect on running tasks

Symptom. A new value is written and the deploy script re-run, but consumers still present the old key — e.g. `/portal/me` returning 401.

Cause. The generated keys (the two spend admin keys, the gateway JWT secret, the portal session secret) are injected as ECS `Secrets` and read **only at container start**. The write happens outside CloudFormation, so the subsequent stack update is an empty changeset and running tasks keep the old value.

Fix. Force a new deployment on every consuming service (`aws ecs update-service … --force-new-deployment`). The spend **read** key has two consumers — the gateway *and* the portal — so roll both. Procedure and the no-argv write pattern: om-runbooks §7, `cost-controls.md` §7.

5.6 Reading the gateway's Postgres from an admin host fails

Cause. Two gates, both by design. **Network:** the database admits only members of the `<prefix>-db-client-sg` security group (stack 01 output `DBClientSecurityGroupId`); security groups do not apply from outside the VPC. **Secret decrypt:** the app-user secret is CMK-encrypted, so the operator role needs `secretsmanager:GetSecretValue` **and** `kms:Decrypt` on the CMK — the `kms:Decrypt` half is the non-obvious one (§2.2).

Fix. Attach the client SG to the in-VPC admin instance's ENI (it is additive) and grant the two permissions. Do **not** widen the DB SG ingress. Details, including the offline `pg8000` install, are in om-runbooks §9a.

5.7 What the tables say when something upstream is broken

- `spend` **empty while inference is happening** → the gateway is metering nothing. Check the gateway log group for `spend meter has no exact rates for model`: the served model IDs are not in the rate table.
- `principal_emails.groups` **NULL or empty** → the Okta groups claim is not arriving (§4.2). The Grafana `user_groups` filter shows nothing **and** per-group spend caps match nobody. Fix at Okta, not in the gateway.
- Postgres holds **aggregate cents per principal per period**, never per-request token counts — those live only in AMP.

---

6. Gateway configuration & managed client policy

6.1 A managed-policy change deploys green and changes nothing

Cause. `GATEWAY_MANAGED_B64` is consumed only by a specific stanza in `docker/entrypoint.sh`. Because ECR tags are immutable and the deploy script defaults the image tag to the configured client version, a running gateway image that predates that stanza decodes the variable with **nothing**. The deploy goes green, the symptom is unchanged, and nothing is logged.

Fix. Confirm the deployed image contains the stanza — or rebuild with a bumped tag — **before** re-running the gateway stack. This applies to every managed-policy change.

6.2 A managed policy silently becomes dead config

Cause. Policy selection is **first-match-wins over a single policy**, and a policy with no `match:` is normalized to `match: {}`, which matches everyone. Put the catch-all first and every later policy becomes unreachable for every user — while the gateway still boots.

The tell in the gateway log:

```
warn managed.policies[0] is a catch-all (match: {}) but is not the last
     entry - policies after it will never match. Move it to the end.
```

With correct ordering the warning disappears and the log instead reports the catch-all's `cli` block merging as a **base** into each earlier policy, so group-scoped members receive both their own policy and the catch-all keys.

Fix. The catch-all allowlist policy must always be the **last** entry. A template test pins the ordering.

6.3 A config probe says the configuration is valid and the gateway still dies on boot

Cause. The `cli:` block is **not** an unvalidated passthrough — it is strictly validated against the client settings schema, and an unknown key is fatal (`managed.policies[N].cli invalid: <key>: unknown settings key …`, exit 1). The catch is *when*: that check runs only **after the Postgres store connects**, so a probe against an unreachable database never reaches it. "Reaches the Postgres error, therefore schema-valid" is a false method for anything inside `cli:`.

Fix. Verify a `cli:` change by booting the mirrored gateway binary against a throwaway Postgres, with a deliberately bogus key as the negative control.

6.4 Gateway boot fails on a config-schema shape

Two shapes that must be right, both invisible to `cfn-lint` because they live inside a base64 config blob (a template test parses that blob and asserts the shape):

- the OIDC issuer must be an `https://` **URL**, not a bare domain — "oidc.issuer must be an http(s) URL";
- `models[].upstream_model` must be an **object keyed by upstream name** (`bedrock: <id>`), not a bare string.

Boot failure here is the fail-closed design working. Read the task's stopped reason and the `/ecs/<prefix>` log stream for the exact message.

6.5 The deploy refuses to run: duplicate gateway-facing model IDs

Cause. A `deploy.env` still assigning an old model ID to a role that a newer default now occupies — the three gateway-facing IDs (`OPUS_MODEL_ID`, `SONNET_MODEL_ID`, `HAIKU_MODEL_ID`) collide.

Fix. `deploy-gateway.sh` fails closed on this deliberately. Update `deploy.env` from `deploy.env.example` — do not override the guard. Verify inference-profile IDs against the region before changing model defaults:

```
aws bedrock list-inference-profiles --region us-gov-west-1 \
  --query "inferenceProfileSummaries[?contains(inferenceProfileId,'anthropic')].inferenceProfileId"
```

6.6 A "no desktop: block" warning at boot

```
no managed policy carries a desktop: block - Claude Desktop clients will be
rejected by /user/bootstrap until a policy opts in
```

Expected and harmless for a Claude Code CLI-only deployment. It would matter only if Claude Desktop came into scope.

---

7. Telemetry pipeline (gateway → sidecar → AMP)

7.1 `ECONNREFUSED_SSRF` on the loopback telemetry forward

Symptom. In the gateway log:

```
forward to http://localhost:4318 failed: ECONNREFUSED_SSRF:
blocked (cloud metadata / link-local): localhost -> 127.0.0.1
```

Metrics stop reaching AMP and the missing-telemetry alarm fires; the gateway itself keeps serving.

Cause. Two gateway behaviours collide. It **refuses a non-HTTPS** `telemetry.forward_to` **unless the host is loopback** — and its SSRF guard **blocks loopback by default** via a custom DNS lookup that rejects any address resolving into a blocked range. Static config validation cannot catch this: the check sees the hostname `localhost`, which does not parse as an IP literal, and only runtime resolution sees `127.0.0.1`.

Fix. `CLAUDE_GATEWAY_ALLOW_LOOPBACK=1` on the gateway container; the template sets it whenever telemetry is enabled. Seeing this error means the running task predates the variable — re-run `deploy-gateway.sh` and confirm:

```
aws ecs describe-task-definition --task-definition <gateway-td> \
  --query "taskDefinition.containerDefinitions[?name=='gateway'].environment[?name=='CLAUDE_GATEWAY_ALLOW_LOOPBACK']"
```

Scope of the override. It re-permits only loopback and unspecified addresses; link-local ranges stay blocked, including EC2 IMDS (`169.254.169.254`), `100.100.100.200` and `fd00:ec2::254`. One benign side effect: the startup "pod can reach cloud metadata endpoint" *warning* is suppressed — a diagnostic, not a control.

7.2 `otelcol_*` lands in AMP but zero `claude_code_*` ever does

Symptom. Collector self-metrics and activity logs flow fine, but no client usage metric ever appears — while every counter looks healthy: receiver-accepted equals exporter-sent, `send_failed` is 0, and nothing is logged at any level.

Cause. Claude Code clients export **delta**-temporality sums by default. The `prometheusremotewrite` exporter cannot represent delta and drops those points at **translation — before the send** — so they are still counted in `sent_metric_points`, `send_failed` stays 0, and nothing is logged. The collector's own self-metrics are cumulative, which is exactly why they land while client metrics vanish.

The one discriminating signal. `otelcol_exporter_prometheusremotewrite_failed_translations` climbing in step with client activity. Check this **first** in any "metrics missing" triage; `scripts/diagnostics/amp-query.py` reads it and verdicts on it first.

Fix. The gateway's managed catch-all policy pushes `OTEL_EXPORTER_OTLP_METRICS_TEMPORALITY_PREFERENCE: "cumulative"` in `cli.env` to every client via `/managed/settings`; clients pick it up at their next settings fetch. Confirm `claude_code_*` names appear and `failed_translations` goes flat.

Do not "fix" it with a `deltatocumulative` **processor.** With more than one gateway task the ALB round-robins one client's exports across both sidecars, and two independent delta→cumulative reconstructions of the same series conflict. Fix temporality at the source.

Collector-version note. The `add_metric_suffixes` deprecation warning on newer collector builds is harmless — the key is still honored. The equivalent replacement is `translation_strategy: UnderscoreEscapingWithoutSuffixes` — the **without**-suffixes variant; the WithSuffixes variant re-adds unit/type suffixes and breaks the dashboard's metric names. It is deliberately un-adopted because an unknown key is a boot failure on older collector pins.

Detection gap. The missing-telemetry alarm is structurally blind to this failure — its heartbeat is the collector's own already-cumulative self-metrics, which keep flowing while 100% of client metrics are dropped. The discriminating counter lives in AMP, which CloudWatch alarms cannot evaluate. Until AMP rule groups cover it, the compensating control is an operator cadence of checking `failed_translations` (§13.5).

7.3 Grafana "Unable to retrieve metric names"; remote-write 403s

Symptom. Grafana reports *"Unable to retrieve metric names / We are unable to connect to your data source (Unknown error)"* over an empty dashboard, the sidecar cannot write, and the missing-telemetry alarm is stuck in ALARM. The 403s are server-sourced and fast, so connectivity is clearly fine.

Cause. The AMP workspace is CMK-encrypted, and AMP charges the KMS crypto operation to the **calling principal** — the `aps.<region>` service grant covers only AMP's internal ingestion, not the data-plane API. Reading (`aps:QueryMetrics` / `GetSeries` / `GetLabels` / `GetMetricMetadata`) needs `kms:Decrypt`; writing (`aps:RemoteWrite`) needs `kms:GenerateDataKey`. See §2.2.

Fix. The Grafana task role holds `kms:Decrypt`; the telemetry sidecar role holds `kms:GenerateDataKey`; both scoped `kms:ViaService=aps.<region>.amazonaws.com` and gated on the CMK-encrypted-workspace case (an AWS-owned-key workspace needs neither).

7.4 Reading a 403 from an AMP query correctly

- Body says `SignatureDoesNotMatch` ("the request signature we calculated does not match") → a client-side **signing or encoding** bug, not a permissions problem. Do not chase key policies.
- A plain 403 **without** that phrasing → an operator-role gap: a missing `aps:QueryMetrics`, or the CMK `kms:Decrypt` trap (§7.3).
- Either way it says nothing about whether **ingestion** is working.

7.5 The missing-telemetry alarm fires every night and weekend

Cause. Client usage metrics are **push-only and bursty** — the gateway forwards them only while a developer is actively using Claude Code. An idle fleet emits nothing, ingestion goes to zero, and the alarm (correctly configured to treat missing data as breaching) fires.

Fix (in place). The sidecar's `prometheus` receiver scrapes the collector's own `otelcol_*` self-telemetry over loopback every 30 s into the same remote-write pipeline. That continuous heartbeat traverses the full SigV4 + KMS + AMP write path, so ingestion is continuous whenever the pipeline is healthy and the alarm becomes a genuine liveness signal rather than a fleet-activity signal. Treating missing data as breaching is load-bearing: AMP *stops emitting* the metric when nothing is ingested, and the CloudWatch default would park a dead pipeline in `INSUFFICIENT_DATA` forever.

Expected, non-actionable firings. Between the observability (03) deploy and the telemetry-enabled gateway (02) re-run, and during deliberate gateway downtime.

Testing it end to end. With the always-on heartbeat the cheapest full test is to stop the sidecar — ingestion stops, the alarm goes ALARM, restart returns it to OK. That exercises IAM, the endpoint, KMS and AMP ingestion in one move.

7.6 "The dashboards are empty" — separating five identical-looking failures

`scripts/diagnostics/diagnose-telemetry.sh` reads the ALB access logs (client → `/managed/settings`, client → `/v1/metrics`) and then queries AMP over SigV4:

Evidence	Meaning
no `/managed/settings` requests	enrollment — clients never got the OTLP env vars
settings fetched but no `/v1/metrics`	push lands, export never starts
exports happening, AMP empty of `otelcol_*` too	remote-write not landing at all (§7.3)
exports, `otelcol_*` present, no `claude_code_*`	check `failed_translations` first (§7.2)
`claude_code_*` present	ingestion is fine — it is a dashboard or query problem

The last row is the easy misread: the dashboard filters on `team`, `cost_center` and `user_groups`, so **absent labels render empty panels over present data**. The script prints the labels actually on `claude_code_cost_usage` for exactly this reason. `team` and `cost_center` come from the installer's `OTEL_RESOURCE_ATTRIBUTES`; `user_groups` is stamped by the gateway from the Okta claim (§4.2). `scripts/diagnostics/amp-query.py` queries AMP directly; its default window (`AMP_QUERY_WINDOW_HOURS`, 48) is long because client metrics are bursty and short windows miss them.

7.7 The first telemetry-enabled deploy never reaches `services-stable`

Symptom. `deploy-gateway.sh` hangs; the only signal is a generic "task not healthy".

Cause. With `TELEMETRY_FAIL_CLOSED=true` the gateway waits on the collector reporting HEALTHY. With `MinimumHealthyPercent: 100` and no deployment circuit breaker, a wrong health-check path or health-check extension configuration hangs the rollout indefinitely.

Fix / prevention. Bring telemetry up **fail-open first** — `TELEMETRY_FAIL_CLOSED="false"` set *in* `deploy.env`, see §1.6 — confirm the `otel-collector` container reports HEALTHY (the same check runs in both postures), then flip to `"true"` and re-run.

Steady-state consequence of fail-closed. A persistently unhealthy collector **stops the gateway task** and ECS replaces it: check the `otel/` log streams and the collector health check before suspecting the gateway itself. Task stops drain gateway-first / collector-last with up to a 120 s flush window, so rotations and deploys take roughly two minutes longer than they otherwise would.

7.8 Do not delete `session.id` from the metric labels

Symptom (if it is ever re-added). Dashboard spend inflates drastically and cost lines show a sawtooth, specifically for developers running multiple concurrent sessions.

Cause. Each session's counters start at 0. Deleting `session.id` for cardinality merges concurrent sessions from the same user and model onto **one** series, where they interleave as a sawtooth; every downward alternation reads as a counter reset and the value is re-added, inflating the total.

Fix (in place). `session.id` is kept as a metric label, so each session is its own monotonic series and the `sum by (…)` panels are exact with no query changes. Cardinality is bounded by *concurrent* sessions — stale series age out of AMP's active-series window within minutes. If a fleet ever grows to where that matters, resize deliberately; re-adding the delete brings the inflation back.

For incident comms. Grafana numbers are observability only. Authoritative spend is the gateway's Postgres `spend` table, which meters inference server-side and is unaffected by label choices.

7.9 Migrating a deployment that still runs a standalone collector service

A deployment predating the sidecar layout has a separate collector ECS service and a Cloud Map namespace in stack 03. The update order is the **reverse** of a fresh deploy — 03 first, then 02 — and one pre-step avoids the most likely failure.

Symptom if the pre-step is skipped. The whole 03 update rolls back: `AWS::ServiceDiscovery::Service` deletion fails while any instance is registered, and ECS deregisters the Cloud Map instance only as the task drains, so CloudFormation can reach the delete first.

Order.

1. Drain the collector service to zero and wait for it — `aws ecs update-service --desired-count 0` + `aws ecs wait services-stable` — then confirm with `aws servicediscovery list-instances`.
2. `deploy-observability.sh` — removes the standalone service, its task and roles, the collector SG and the gateway↔collector rule pair, and the namespace; adds the AMP outputs and the missing-telemetry alarm.
3. `deploy-gateway.sh` — attaches the sidecar with the new AMP parameters. Running 02 first would point it at an endpoint SG rule 03 has not rewired yet.

Between steps 2 and 3 the old gateway's `forward_to` resolves to a deleted name. That is benign: forward failures are non-fatal, and the off-localhost forward never worked (§7.1 is why the sidecar exists).

---

8. Grafana & dashboards

8.1 After a Grafana major upgrade, the AMP datasource has no SigV4 option

Cause. Grafana ≥13.1 **removed SigV4 auth from the core prometheus datasource**; AMP auth moved to the Grafana-signed `grafana-amazonprometheus-datasource` plugin, which is not bundled.

Fix. The plugin is baked into the image — an egress-less task cannot fetch it at boot — pinned and sha256-verified in `scripts/mirror/grafana-plugin.pin` (bump `AMP_PLUGIN_VERSION` and `AMP_PLUGIN_SHA256` together), staged by `scripts/mirror/mirror-grafana-plugin.sh` on the egress host, and provisioned as that datasource **type**. The datasource uid is unchanged, so dashboards are unaffected.

Three companion traps on any Grafana version bump.

- The maintained OSS image repository is `grafana/grafana`; the `grafana/grafana-oss` Docker Hub repository froze at 12.4.
- Grafana ≥12 runs a **background plugin preinstaller that dials grafana.com at every boot** — a startup-egress crash loop on an egress-less task. `GF_PLUGINS_PREINSTALL_DISABLED=true` is required. `GF_PLUGINS_PUBLIC_KEY_RETRIEVAL_DISABLED=true` is also set; confirm the plugin's signature still reports valid offline.
- **A digest-pinned `GRAFANA_BASE_IMAGE` in `deploy.env` beats the build script's `GRAFANA_VERSION`-derived default** — bumping only the tag rebuilds the OLD Grafana under a NEW name. Start a version bump with `GRAFANA_VERSION=<new> scripts/mirror/mirror-base-images.sh grafana`, carry the updated `GRAFANA_BASE_IMAGE` line to the build host, then build.

Expected user-visible effect. A one-time re-login for all Grafana users on the first post-upgrade start (external OAuth sessions are re-linked).

8.2 A panel errors: "vector cannot contain metrics with the same labelset"

Symptom. Intermittent, depending on what the fleet did — historically the Sessions and Active-users tiles.

Cause. A selector like `{__name__=~"claude_code_.+"}` inside a range function drops the metric name, so two different metrics from one session with otherwise identical label sets collide. Live, that is any session incrementing two attribute-less counters; cost and token series escape only because their `model` / `type` labels differ.

Fix. Select a single metric (`claude_code_cost_usage`) for distinct-count panels. The same shape in `scripts/diagnostics/amp-query.py` uses the series endpoint instead of an identical range query.

8.3 A usage panel misreports — five distinct shapes

The dashboard's cumulative time-series compute a **per-session in-range rise**: the counter's peak within the range, minus the session's value at the range start when it was already running (baseline looked up to 7 days back), else the full counter; a baseline *larger* than the in-range peak means the counter reset, and the expression falls back to the peak alone. The burn-rate panels compute a **per-session in-window rise** over the trailing hour. Five failure modes are fixed in the shipped expressions; know the shapes, because they recur whenever the expressions are edited.

1. **A session disappears from the graph an hour after the developer stops.** A trailing-1h window drains a session's contribution within an hour of the client going quiet, so spend appears to vanish from the right edge. *Fix:* the **Cumulative (selected range)** panels hold each session's final value to the right edge. The trailing-1h panels are kept, retitled **Burn rate**, where the drain-off is the intended reading ("who is spending right now").

2. **A ghost session at full value at the left edge that "drops off" mid-graph.** Range-width-dependent — a 2-day view ghosts a session that ended before the range start while 1-day and 3-day views of the same data are clean. Any session that ended within one range-width *before* the range start still sits inside the per-plot-point `[$__range]` lookback while its `offset $__range` baseline window is empty, so the whole counter is counted. *Fix:* gate each cumulative series on having a sample **inside the visible range** — `and count_over_time(m[$__range] @ end())`; the `@ end()` window equals the visible range at every plot step.
3. **A curve that steps DOWN near the right edge.** For sessions that started *before* the range start; a 12h view declines while the 24h view of the same data is clean, and the decline mirrors the session's early climb one range-width later. A `last_over_time(m[1h] offset $__range)` baseline is evaluated **per plot point**, so the window slides forward and grows as it crosses the session's pre-range climb — a growing subtrahend. *Fix:* anchor the baseline at the visible range start — `last_over_time(m[7d] @ start())` — a fixed window identical at every step, so each curve is exactly `counter(t) − counter(range start)`: monotonic, with the same right edge.
4. **Yesterday's spend attributed to today by an idle-resumed session.** A developer runs up spend, leaves the session open overnight (or over a weekend), and resumes: the day view shows the full prior counter as a plateau from midnight, a dip to zero mid-morning (the old samples slide out of the per-step lookback before the new ones begin), then a jump to the counter's resumed value — the prior day's spend counted again, in tiles and curves alike. With a 1-hour baseline window, any session silent for >1h across the range-start boundary had an *empty* baseline and counted its full counter. *Fix (two-part, in the same per-series core):* the baseline looks back **7 days** (`last_over_time(m[7d] @ start())`, `offset $__range` on the instant tiles), and the missing-baseline fallback is the filter form `((peak − baseline) >= 0) or peak` — replacing the old `(0 + baseline) or (0 * peak)` arithmetic. The filter form is what makes the long baseline safe: a client that restarts and resumes the *same* session id re-exports its counter from zero, the difference goes negative, the `>= 0` drops it, and the `or` counts the fresh counter instead of a negative contribution (a zero rise passes `>= 0` and correctly stays zero). The fallback is exact only while the fresh counter stays *below* the pre-reset baseline; once it overtakes it the filter branch resumes and the contribution becomes peak − baseline — an under-count of at most the pre-reset spend, with a one-time step down in the curve at the crossing (see the accepted caveats below). Both parts verified against a backfilled engine with idle-resumed, fresh, ended-pre-range, and counter-reset sessions; `tests/templates/test_usage_dashboard.py` pins the shape.
5. **A burn-rate panel spikes by a session's full counter value, for exactly one hour.** A client restart that resumes the same session id re-exports its counter from zero *while the session is actively reporting*; a trailing-window `max_over_time − min_over_time` then reads pre-reset peak minus post-reset minimum — approximately the whole counter — at every step whose window straddles the reset, i.e. for exactly one window-width. The fingerprint is the duration: the spike holds flat for precisely 1h, then collapses. The discriminator from shape 4: the **cumulative** panels stay clean across the same event (there a reset is a bounded under-count, shape 4's fallback note), so a burn-rate-only spike means a reset, not a baseline problem. *Fix:* `last_over_time − min_over_time`. While the counter is monotonic within the window the last sample *is* the max, so the reading is identical to the old form everywhere it was right; on a reset window it yields the post-reset rise (last ≥ min structurally, so never negative and never the pre-reset total). Verified against the same backfilled engine: the reset scenario's hour-long full-counter spike becomes the true post-reset rise, all other scenarios byte-identical; `tests/templates/test_usage_dashboard.py` pins this shape too.

Why tiles were always right. Tiles and the top-users table run as **instant** queries: at instant evaluation the `offset $__range` window already is the anchored window and the `@ end()` gate is a no-op. Running them as range queries would evaluate the full-range expression at every plot step and keep only the last point — pure wasted AMP cost.

Invariants to re-check after any dashboard edit. The cumulative right edge equals the stat tiles; curves never step down (one exception: a reset session steps down once when its fresh counter overtakes the pre-reset baseline); a finished session is still visible at the right edge an hour later; a session that ended just before the range start does not appear; a session idle overnight that resumes contributes only its new spend; no series ever contributes a negative value; a burn-rate curve never exceeds real in-window spend (a reset shows the post-reset rise, not the pre-reset total).

Multi-day ranges depend on `@` rewriting. AMP accepts the `@` modifier. Multi-day ranges additionally rely on the query frontend rewriting `@ start()` / `@ end()` to absolute timestamps *before* its per-day split. Probe it with a `query_range` over a ≥3-day window of `last_over_time(claude_code_cost_usage[7d] @ start())` through `scripts/diagnostics/amp-query.py`: every returned series must be a **flat constant** across the whole range. A value that steps at each 24 h boundary means the frontend resolved `start()` per sub-query, and the rollback is to drop back to `offset $__range`. A missing `@` implementation fails **loudly** with a parse error, never with silently wrong data.

Cost shape. Full-range lookbacks at every plot step are the expensive query shape on AMP, whose query frontend splits long range queries into per-day sub-queries that each refetch the whole lookback. Curves are capped at `maxDataPoints: 200`. The 7-day baseline window widened this further (from 1h) — the window is anchored, so it scans the same absolute span at every step, but it does scan a week of every matching series per panel refresh; if AMP query cost becomes a concern, the window is the knob, at the price of re-widening the idle-resume hole to whatever it is cut to.

Remaining accepted caveats. (1) A session already running before the range start whose samples go silent for more than **7 days** across the boundary has an empty baseline window and counts its full counter, pre-range spend included — the shape-4 misattribution, pushed out from

> 1h of silence to >7d. (2) A same-session-id counter reset whose fresh counter overtakes the pre-reset value is under-counted by at most that pre-reset value (shape 4's fallback note above); the error is bounded, never negative, and never an over-count. (3) In the burn-rate panels a reset window drops the session's pre-reset in-window rise (only the post-reset rise is measurable from the window's samples) — an under-count of at most one hour of that session's real spend, gone once the reset leaves the window.

Deploying a dashboard change. The dashboard JSON is baked into the Grafana image: bump `GRAFANA_IMAGE_TAG`, rebuild and push, re-run the observability stack (om-runbooks §6).

8.4 Dashboard numbers disagree with the bill

Dashboards are **observability**. For exact spend accounting the gateway's Postgres `spend` table is authoritative (`cost-controls.md` §3.3). Dashboard history predating the `session.id` and window-function reworks (§7.8) is unreliable and should not be used for retrospective accounting.

---

9. Client (Claude Code on the laptop)

End-user-facing versions of these live in `client-config.md` §5; this section is the operator's view. The enforcement model itself is that document's Part II.

9.1 There is no "Cloud gateway" login option at all

Cause. `forceLoginMethod: "gateway"` and `forceLoginGatewayUrl` are honored **only** from a managed source — `HKLM\SOFTWARE\Policies\ClaudeCode`, `%ProgramFiles%\ClaudeCode\managed-settings.json`, or Linux `/etc/claude-code/managed-settings.json` — never from user settings or HKCU. This is anti-phishing design: there is no user-scope substitute and no place to type a gateway URL.

Fix. Deliver the managed settings by GPO/MDM (`client-config.md` §8, and `docs/requests/ad-request-email.md` for the request), or self-serve once on a machine with local admin or sudo. For a single-laptop bring-up, deliberately **omit** `forceRemoteSettingsRefresh`: that key makes the CLI exit if it cannot fetch the gateway's managed settings — the production posture, but it leaves no working CLI to debug with while the gateway is still coming up. Add it once login works, so the machine matches the fleet; the `/model` check is what confirms the push landed.

9.2 Startup fails: "Unable to connect to Anthropic services"

Symptom. `claude` exits at launch before any login screen, and `claude auth login` refuses with a message about `forceLoginMethod` being `gateway`. Typically right after `/logout`, or on a **first-ever run on a clean profile** whose install predates the installers setting `hasCompletedOnboarding` at install time (current installers set it — but the portal serves the installers last uploaded by `publish-portal-release.sh`, so this can persist until a re-publish).

Cause. The two logout paths are not equivalent. `/logout` (the slash command) clears credentials **and** onboarding state and deletes the whole credential store including the pinned gateway trust; `claude auth logout` (the CLI subcommand) clears credentials only. With onboarding unset, Claude Code derives the login method from whether gateway credentials exist, resolves to first-party, and runs a **connectivity preflight** against Anthropic's public endpoints before drawing the gateway login screen. On a gateway-only egress path that is a deadlock.

Fix. Recovery is a one-line local config change made **as the affected user, no admin required** — the exact procedure, the confirming curl checks, and the service-desk version are om-runbooks §12 and `client-config.md` §5.6. Three things not to do: `forceRemoteSettingsRefresh` is **not** the culprit and must not be stripped from the GPO during triage (that would silently drop the model allowlist fleet-wide); opening Anthropic's public endpoints in the egress policy also "fixes" it and permanently widens egress to work around a local config; and the returning TLS fingerprint prompt must be **checked against the published value**, not clicked through — that prompt is the interception control.

Prevention. Tell developers to sign out with `claude auth logout`, never `/logout`. Because the same trap applies to a fresh install on a clean profile, exercise a never-used profile before broad rollout.

9.3 Developers bounced to full browser SSO every hour

Symptom. Browser SSO at a fixed interval matching `SESSION_TTL_HOURS`, often paired with: after the bounce, `/login` shows the default account picker instead of the gateway, and only restarting Claude Code restores the forced-gateway login.

Cause. The gateway refreshes its session JWT using an **upstream Okta refresh token**, and Okta issues one only when the app has the **Refresh Token grant type** *and* `offline_access` granted. Clients request `offline_access`, but an app configured for Authorization Code only makes Okta drop it silently — so there is

nothing to refresh and the session dies at the TTL. The picker follow-on is a consequence: `forceLoginMethod` is applied at process startup and the mid-session re-login path does not re-assert it.

Fix (Okta admin, no redeploy). Enable the Refresh Token grant alongside Authorization Code, confirm `offline_access` is granted, and on an org authorization server confirm its refresh-token policy issues them. Then log in fresh once. This is TTL-independent — lowering `SESSION_TTL_HOURS` only changes revocation propagation speed. Success logs `[gateway-refresh] refreshed gateway JWT`; failure logs `[gateway-refresh] IdP rejected refresh token; clearing it` or `OAuth session expired and could not be refreshed`.

9.4 `/model` shows Claude Code's built-in menu and every pick fails

Cause. The client picker is constrained only by `availableModels` / `enforceAvailableModels` pushed through `/managed/settings`; the gateway's own `models:` block governs only what the **gateway serves**, so an unconstrained picker offers models that then fail upstream as unauthorized.

Checks, in order.

- If the built-in menu still appears after a policy change, the running gateway image predates the `GATEWAY_MANAGED_B64` stanza and ignores the variable **silently** — §6.1.
- A catch-all-ordering warning in the gateway log means the allowlist policy is shadowing the group-scoped policies — §6.2.
- `availableModels` must sit **inside the policy's** `cli:` **object**; at policy level it is an unrecognized key that fails config validation and prevents boot — §6.3. `enforceAvailableModels` extends the allowlist to the "Default" selection so Default cannot resolve to an unserved model.

9.5 Background / small-model calls fail while normal requests work

Cause. Claude Code uses a Haiku-family model for lightweight background work by default. No Haiku-family model exists in GovCloud and this gateway serves none, so those calls request an unserved model.

Fix. The catch-all policy sets `env.ANTHROPIC_DEFAULT_HAIKU_MODEL` to the configured haiku-role model ID. Two gotchas: the value must be the **gateway-facing** ID, not the Bedrock inference-profile ID (the gateway's `models:` block does the Bedrock mapping); and `ANTHROPIC_SMALL_FAST_MODEL` — the deprecated name for the same knob — **takes precedence if a user has set it locally**. The `/model` picker may also grow a cosmetic custom-Haiku entry; it resolves to an allowlisted model either way.

9.6 A machine still forces an old gateway URL, or refuses an approved build

Cause. An earlier installer wrote forced-login keys and `requiredMinimumVersion` to a **managed** source — `HKCU\SOFTWARE\Policies\ ClaudeCode` on a non-admin run, or a machine-scope `managed-settings.json` when elevated. Managed sources override user scope, so they keep taking effect. The current installer writes nothing to managed sources and therefore does not clean them up.

Fix. Remove the stale HKCU key and both managed-settings paths — unless you are deliberately taking them over via GPO/MDM — then confirm with `/status` inside `claude` that no unexpected managed source remains. Procedure: `client-config.md` §8.4.

9.7 Clients exit at startup below the required minimum version, with no way to comply

Cause. `deploy-gateway.sh` defaults the pushed `requiredMinimumVersion` to `CLAUDE_VERSION`, and client auto-updates are locked down — the portal is the only update path. Raising the gateway's floor before the matching installer is published locks the fleet out.

Fix / **order.** Publish the installer with `publish-portal-release.sh` **before or with** the gateway roll, or pin `MIN_CLIENT_VERSION` in `deploy.env` to hold the floor while the fleet catches up (`none` disables the check). Two related traps:

- **Rollback.** Reverting to a previous image requires reverting `CLAUDE_VERSION` (or pinning `MIN_CLIENT_VERSION`) **in the same edit** — restoring only the image URI leaves the next deploy pushing the new floor while the portal and fleet are back on the old version.
- **Do not set the floor in two places.** Which managed source wins when both a GPO JSON and the gateway push set a floor is undefined. Keep the key out of the GPO copy unless deliberately managing it there — and then set `MIN_CLIENT_VERSION=none`.

"Publish, then raise" is the general shape for anything the client must comply with: installer before version floor, fingerprint before certificate cutover.

9.8 The installer refuses to run

A SYSTEM-context run is **refused outright** — it would install the binary into SYSTEM's profile and PATH, which no developer ever sees. There is no settings-push mode either: the installer writes only the user settings `env` block, never a machine or policy source. For a device-context binary push, use the MDM "user" install behaviour or the download portal.

---

10. Cost controls & spend caps

10.1 Fleet-wide 429s, including users with no cap

Symptom. Every user gets HTTP 429 simultaneously; `spend check failed` / `store_error` in the gateway log group; often correlated with RDS trouble or a recent credential rotation.

Cause. `enforcement.fail_closed_on_error: true` — a deliberate availability trade. If the spend store is unreachable the gateway refuses all inference rather than allow uncapped spend. **No dedicated CloudWatch alarm watches this condition**; detection is user reports plus the DB-side alarms.

Fix. Fixing the database is the real fix — enforcement recovers on its own. The full incident runbook, including the break-glass flip and its consequences (spend becomes unmetered and uncapped), is `cost-controls.md` §5. Triage order: scope (one user capped is normal enforcement; everyone is an incident) → gateway

logs for `store_error` → RDS status, storage and connection exhaustion, plus the `<prefix>-db-rotation-errors` alarm (a botched app-credential rotation looks exactly like a store outage, §5.3) → DB SG membership and gateway task health.

10.2 A single user gets 429 and waiting does not help

Cause. Normal cap enforcement: HTTP 429, error type `billing_error`, message *"spend limit reached — "*, and `x-should-retry: false`. It is **not transient** — the client will not retry, and nothing changes until the period rolls or the cap does.

Fix. `SPEND_BLOCKED_MESSAGE` is the only self-service breadcrumb the developer sees; set it to org routing text. Confirm the diagnosis with `set-spend-limit.sh --list` plus `dump-usage.sh` (spend row versus cap), and lift with a per-user cap — per-user always wins, and in the default group mode a generous group cap cannot loosen a tighter one. `cost-controls.md` §4.

10.3 Portal admin page: "the gateway refused: your account is not in its spend-admin groups"

Cause. `SPEND_ADMIN_GROUPS` (stack 02, which feeds the gateway's `admin_groups`) and `PORTAL_ADMIN_GROUP` (stack 04, which gates the page's visibility) have diverged.

Fix. Re-align them and re-run the affected deploy script. An empty `SPEND_ADMIN_GROUPS` disables bearer-token admin entirely — only the generated keys work; an empty `PORTAL_ADMIN_GROUP` hides the admin page. Note that this failure also has an identity cause: if the groups claim is not arriving at all, see §4.2.

10.4 A "cleared" user is UNLIMITED instead of falling back to the org/group cap

Symptom. A user whose cap was cleared shows unbounded spend; the caps list (portal grid or `set-spend-limit.sh --list`) still shows their row, with a null amount.

Cause. A cap row with `amount: null` is not "no cap" — it is an **explicit unlimited override at that scope** and, for a user row, it wins over every group and org cap (user precedence). Binary-verified 2.1.220 and observed live 2026-07-29. Pre-2026-07-29 portal "Clear" and `set-spend-limit.sh --clear` created exactly these rows by POSTing `amount: null`; both now DELETE the row instead, and the portal renders surviving null rows as *UNLIMITED override*.

Fix. Remove the row (portal **Remove** button, or `--clear`, both of which now issue `DELETE /v1/organizations/spend_limits/<id>`); the user falls back to group/org caps immediately. After upgrading, sweep `--list` for `"amount": null` rows — each one is an active exemption. Related authorization behaviour worth relying on when triaging: the **read key is refused for writes** (403), an unauthenticated call is 401, and a valid token whose identity is not in the admin groups is 401 on read *and* write.

10.5 A user-scope cap never applies (set by email or bare sub)

Symptom. A per-user cap is set and listed, but the user's spend sails past it; the All-users page shows their cap source as the group/org cap (or none), not the user cap.

Cause. The gateway matches user caps by **exact string equality against the principal** `oidc:<sub>` — nothing else. The admin API accepts any string as `user_id` and returns success, so a cap keyed by the user's email or bare Okta sub is stored but matches nobody, with no error anywhere. Binary-verified 2.1.220, confirmed live 2026-07-29 (pre-fix portal and docs wrongly said "sub or email").

Fix. Since 2026-07-29 the portal and `set-spend-limit.sh` resolve an entered email to the principal before writing, and the portal refuses a bare sub outright (the user must have signed in at least once for email resolution; an id already in `oidc:` form is written verbatim — in orgs whose Okta subs are emails, principals legitimately contain `@`). The portal grid flags any user row not in `oidc:` form with "never matches a user". Remove such dead rows (portal Remove, or `--clear` — the entered email matches the legacy row first) and re-set the cap by email (now resolved) or by the `oidc:<sub>` shown on the All-users page.

---

11. Download portal

11.1 A platform's download aborts mid-stream

Cause. A portal image that offers a platform, served against an artifacts bucket published by an older publish script, streams release objects that do not exist.

Fix / **order.** Re-run `publish-portal-release.sh` **before or with** the portal image bump. The publish script requires, re-verifies and uploads every platform binary and installer.

11.2 An admin is bounced back to "connect" right after a successful grant

Symptom. The login loops with no diagnostic message.

Cause. Browsers **silently discard** an oversized `Set-Cookie` (roughly a 4 KB per-cookie cap). A large Okta groups claim pushes a session or token cookie over the limit.

Fix. The cookie budget is explicit: the refresh token is dropped first, and an explicit error is rendered rather than a silent loop. The budget applies to **every** cookie that embeds the groups list. If this recurs, the groups claim is the thing to shrink.

11.3 The portal task fails to boot after a dropdown configuration change

Cause. A malformed `PORTAL_COST_CENTER_TEAMS` mapping. The task fails at boot **by design** with a clear error rather than rendering an empty dropdown. Reserved delimiters (`, : |`) are rejected inside values, alongside the OTEL no-space, no-comma rules; validation is on the (team, cost-center) **pair**, not two independent list memberships.

Fix. Correct the mapping in `deploy.env` and re-run `deploy-download-portal.sh`. A deployment still carrying an older, flat team/cost-center configuration fails earlier and more loudly: the script's `require_vars` lists `PORTAL_COST_CENTER_TEAMS` as mandatory, so an unset mapping stops the deploy before the stack is touched.

11.4 The cost-change audit trail shows only opaque subject ids

Cause. The gateway's `admin_audit` table records `oidc:<sub>`, and its schema belongs to the gateway binary — it cannot grow an email column.

Fix (in place). Identities are resolved by joining alongside: the portal persists a sub→email map at admin connect (attribution only, never authorization), the portal audit page adds an Email column, portal-side audit lines carry `gateway_actor`, and `dump-usage.sh` LEFT JOINs the gateway's own `principal_emails`. Unmapped actors — pre-feature rows, admins who never connected through the portal, break-glass keys — render as a dash, which is expected, not a fault. Full description: `cost-controls.md` §7.

11.5 The All-Users "% used" meter bears no relation to the percentages

Symptom. On `/portal/admin/users` (portal image ≤ 2.0.0), the small usage bar under each percentage renders at essentially the same length for 0.4% and 72%, and a 35% row can render *longer* than a 72% one.

Cause. A CSS class collision, not bad data: the flash-notice rule `.ok` (padding + border) also matched the green fill span's `ok` status class, adding a constant ~26px to every sub-70% bar regardless of its width class. The percentages themselves — text, sorting, `/portal/me` figures — were always correct.

Fix. Upgrade the portal image (bump `PORTAL_VERSION`, rebuild+push, re-run `deploy-download-portal.sh`). The flash rules are now scoped to their `<p>` elements, and the column renders as a proportional background fill of the "% used" cell itself (a status-colored wash up to the percentage, with a full-strength edge marker) instead of a separate mini bar. `tests/portal/test_static_css.py` gates the width ladder and the bare-status-selector collision so the class of bug can't silently return.

---

12. Bedrock prompt logging

12.1 Enabling fails with a misleading S3 bucket-policy error

Symptom. `PutModelInvocationLoggingConfiguration` fails at the very end with *"Failed to validate permissions for bucket … verify the S3 bucket policy"* while the bucket policy is provably correct.

Cause. The denial is at **KMS**, not S3. Bedrock's enable-time test write to the SSE-KMS bucket needs `kms:GenerateDataKey` for `bedrock.amazonaws.com` **on the key policy** — a service principal cannot use a key through IAM identity policies alone — and Bedrock's error text blames S3 regardless.

Fix. Read the actual CMK key policy for the `BedrockInvocationLogsWrite` statement. Stack-managed key → re-run `deploy-database.sh` for an in-place key-policy update. Bring-your-own **or stack-detached** key (§2.1) → apply the statement out-of-band with get → append → put, preserving existing statements; if the deploy role lacks `kms:PutKeyPolicy` this becomes a request to the key admins. `deploy-observability.sh` preflights this and fails fast with the real cause. Full procedure and the enable order: om-runbooks §11.

12.2 A prompt-log entry looks empty

Not a fault. The CloudWatch leg only ever carries bodies up to 100 KB. An entry carrying an S3 reference instead of a body is **normal** for large Claude Code contexts — the S3 copy is the only place large bodies land.

12.3 Blast radius, before enabling or disabling

Model invocation logging is an **account + region** Bedrock setting. It captures every Bedrock invocation in the account, not only this gateway's, and disabling it disables it for everything. It also does **not** attribute prompts to developers: the log identity is the gateway task role, and per-user attribution remains the activity stream.

---

13. Alarms & monitoring

The alarm inventory and the per-alarm triage order live in om-runbooks §9; `ALARM_SNS_TOPIC_ARN` routes them. This section is the "what does this alarm actually mean" index.

13.1 `<prefix>-missing-telemetry`

Ingestion into the AMP workspace has stopped for `MISSING_TELEMETRY_ALARM_MINUTES`. It is the end-to-end backstop for the fail-closed telemetry posture — container health only proves the collector is alive, this proves data is landing. Triage: om-runbooks §9. Causes covered here: §7.1 (loopback forward), §7.3 (caller-side KMS), §7.5 (expected firings), §7.7 (fail-closed rollout).

13.2 `<prefix>-missing-activity-logs` (off by default)

Watches the *audit* pipeline, which the telemetry alarm and the collector health check cannot see. **Before declaring a fault, confirm the fleet was actually active** — the activity stream is intermittent, so genuine idleness reads as silence. Correlate: a live metrics stream plus a dead audit stream is a real fault; both quiet is probably idle. It is off by default for exactly this reason — enable it only on continuously-active fleets, with a window longer than the longest expected quiet gap.

13.3 `<prefix>-db-rotation-errors`

Running tasks are still fine on the current credential; the rotation SLA is what is at risk. See §5.3. The threshold deliberately tolerates the one expected `Inactive`-image error per scheduled rotation.

13.4 `<prefix>-certificate-expiry`

The imported ACM certificate is approaching expiry and will **not** auto-renew. Execute om-runbooks §1, and publish the new fingerprint before cutting over (§3.9).

13.5 Known blind spots

- **Alarms with no SNS topic.** If `ALARM_SNS_TOPIC_ARN` is empty the alarms still exist and show state in the console, but **nobody is notified**. A subscribed topic is an organizational prerequisite for unattended operation.
- **Client-metric loss is invisible to the telemetry alarm** — the heartbeat keeps the pipeline "healthy" while 100% of client metrics are dropped at translation (§7.2). Compensate with an operator cadence: check `otelcol_exporter_prometheusremotewrite_failed_translations` via `scripts/diagnostics/amp-query.py`.
- **The fail-closed spend store has no dedicated alarm** (§10.1); detection is user reports plus the DB-side alarms.
- **No account-level dollar backstop exists.** The alarms watch pipeline health, not spend. See `cost-controls.md` §6 for the recommendation and why it is left to org policy.

---

14. Offline build & mirroring

14.1 A build script "usually works" and fails on the real build host

Cause. The build/deploy machine reaches **only AWS service endpoints** — not Docker Hub, not grafana.com, not the Claude release CDN, and not AWS-hosted public *download* endpoints either (the RDS truststore, `public.ecr.aws`), which are CDN endpoints rather than VPC-endpoint-reachable services. A fetch that works wherever egress happens to exist is a latent failure that surfaces only on the hardened host.

Fix. Everything external flows through `scripts/mirror/*.sh` on a separate **egress host**; those scripts verify (sha256/GPG) and stage into `mirror/`, which is then copied to the build machine. Build scripts **consume** `mirror/` — `require_mirrored_file` fails closed with transfer instructions naming the exact mirror script to run — and must **never invoke a mirror script**, which would reintroduce the network dependency the mirror exists to remove. Base images are mirrored into the deployment's own ECR, digest-pinned, by `scripts/mirror/mirror-base-images.sh`; upstream defaults exist for dev convenience only.

14.2 `COPY --chmod` fails during an image build

Cause. `COPY --chmod` is a BuildKit feature; hardened hosts often run the legacy Docker builder, frequently with a restrictive umask.

Fix. Use `COPY --chown` plus an explicit `RUN chmod`.

14.3 `update-ca-certificates` warns "does not contain exactly one certificate or crl"

Cause. Debian's `update-ca-certificates` hands its directory to `openssl rehash`, which refuses multi-certificate files — so a concatenated extra-CA bundle staged as a single `.crt` is skipped for hashing.

Impact. Cosmetic-plus: the certificates still reach `/etc/ssl/certs/ca-certificates.crt` (the bundle Python's `ssl` module reads), `update-ca-certificates` exits 0, and the build succeeds. Only the `/etc/ssl/certs` hash symlinks, used by capath consumers, are missing.

Fix. Split the staged bundle one certificate per file before running `update-ca-certificates`.

Two warnings this does not remove, both harmless. A `rehash: warning: skipping ca-certificates.crt …` line appears on every run even with a single perfect certificate (rehash scans the master bundle), and newer Debian bases emit a different message set than older documentation describes. **If a build or deploy actually aborted, the fatal error is elsewhere in the log — these warnings never fail a build.**

14.4 A wrong-architecture image is stuck in an immutable repository

Cause. Pushing a wrong-architecture image under a fixed tag into an IMMUTABLE repository both blocks the (fail-closed) service from starting and cannot be corrected by re-running the mirror.

Fix. Recovery is a manual `batch-delete-image` plus a re-mirror. Prevention: mirror pulls pin `--platform linux/amd64` and assert the architecture after pulling. See §1.7 for the general immutable-tag rule.

14.5 `deploy.env` values do not appear on the build host

Cause. `set_env_var` persists to the **local** `deploy.env` only. When the egress/mirror host is not the build machine, the two files diverge.

Fix. Copy the persisted `*_BASE_IMAGE` and `COLLECTOR_IMAGE` lines across by hand — the mirror scripts print a reminder saying so.

14.6 The release mirror refuses to start

Cause. Verification fails closed by design: without `ANTHROPIC_GPG_KEY` (or the deliberate, named `ALLOW_UNVERIFIED_MANIFEST=1`) the very first mirror step stops.

Fix. Supply the key. If the override is used instead, record that choice — it is auditable, and it must never become the default. Keep this pattern for any new integrity check.

---

15. Teardown & re-create

The teardown procedure, the protection flags to clear, and the full list of what survives are om-runbooks §13. Two symptoms are worth indexing here.

15.1 A stack delete fails

Causes, in the order they appear.

- **Protection flags.** RDS `DeletionProtection` blocks deleting the database stack; ALB `deletion_protection.enabled` blocks deleting the gateway stack. Disable each before the delete. The stack policies deny `Update:Replace` / `Update:Delete` during *updates* only — they do not block `delete-stack`.
- **Export locks.** A stack that another stack still imports from cannot be deleted: the order is `04` and `03` → `02` → `01`.
- **Lingering ENIs.** The db-admin Lambda ENIs can stay attached to the database stack's client SG for roughly 20 minutes; a delete failing with a dependency violation usually just needs a wait and a retry.

15.2 An immediate redeploy of the same prefix fails

Causes. Named Secrets Manager secrets enter a 7–30 day recovery window, and retained log groups collide on re-create (§1.1).

Fix. `aws secretsmanager delete-secret --force-delete-without-recovery` on the `<prefix>/*` secrets, and delete the retained log groups first. Also decide deliberately about the retained CMK: `deploy.env` still holds its ARN in `KMS_KEY_ARN`, so a fresh deploy consumes it as **bring-your-own** and the new stack would reference the key but never manage its policy (§2.1). Either clear `KMS_KEY_ARN` so the new stack creates and manages a new key, or keep it and own the key policy out-of-band from then on. Retained resources are **not** free.

15.3 Restoring a database is not an update

A replaced RDS instance is an **empty** database, and the DB endpoint is a cross-stack export locked while imported — the stack cannot be pointed at a snapshot in place. Restore the snapshot out-of-band to validate the data first; making it live means an orchestrated teardown of the gateway stack, a restore of the database stack from the snapshot, then a redeploy of the downstream stacks. Plan it as a maintenance-window operation: om-runbooks §8.